USER MANUAL

Fei Guo[1], Baihan Li[1], Zhiyu Zhang[1]*, Xiaohong Zhang[1,2]

[1] School of Geodesy and Geomatics, Wuhan University, Wuhan 430079, China

[2] Chinese Antarctic Center of Surveying and Mapping, Wuhan 430079, China

[*] Corresponding author: Zhiyu Zhang; E-mail address: zyzhang_sgg@whu.edu.cn

This paper introduces an open-source software package for sea surface height (SSH) monitoring, utilizing Global Navigation Satellite System Interferometric Reflectometry (GNSS-IR) technology. The software package, developed in MATLAB, is capable of processing signals from four GNSS constellations, namely GPS, GLONASS, Galileo, and BDS. The software consists of three main modules: reflected signal extraction, reflector height calculation, and high-resolution sea level estimation. Additionally, a graphical user interface (GUI) is developed using MATLAB App Designer.

Installation

The software was developed in MATLAB R2021a and is compatible with this version and subsequent releases. The software package is available at https://github.com/WHUSGGzzy/GNSS-R_Toolbox.

Download and extract the compressed files, which will yield two folders (*GNSS-R* and *Data*) and this user manual (*usermanual.pdf*), as shown in Figure 1.

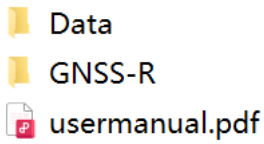

Figure 1. Folders and files from the decompression file.

The '*GNSS-R*' folder (see Figure 2) contains the MATLAB scripts, GUI file ('*GNSSR.mlapp*'), and other files. '*Data*' folder (see Figure 3) includes the test datasets and reference results for validation.

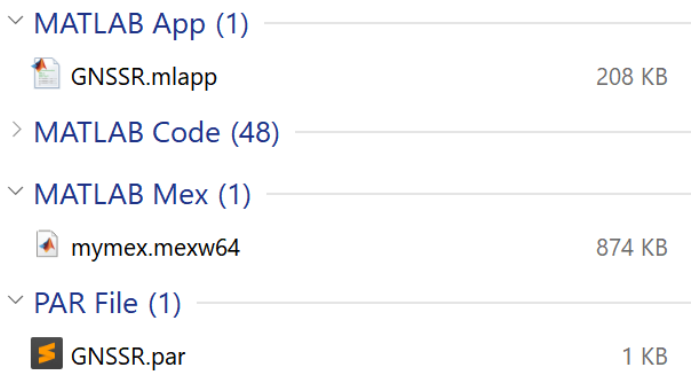

Figure 2. Folders and files in the '*GNSS-R*' folder.

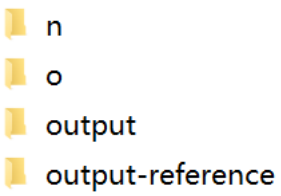

Figure 3. Folders in the '*Data*' folder.

The preparation involves creating three empty folders: the '*o*' folder is used to store observation files, which need to be in RINEX 3 format; the '*n*' folder is for navigation files, which requires precise ephemeris; and the '*output*' folder is for storing result files.

We provide a 7-day test data sourced from the BRST station, and the test data are obtained from The International GNSS Service (IGS). As shown in Figure 3, three folders have been created in the '*Data*' directory as previously described. The observation files and navigation files have been placed in the '*o*' folder and '*n*' folder, respectively. The '*output-reference*' folder provides reference results which could be used to verify the output files.

Figure 4 displays the observation files and navigation files included in the test data. The observation files should follow the short file naming convention, and user could convert long filenames to short filenames (for example, '*BRST3520.21o*'). For navigation files, the software supports the '**.sp3*' format by default, and users can also modify the software to accommodate different formats.

Once data preparation is complete, users can proceed with the

subsequent steps of the software.

BRST3520.21o	22,486 KB
BRST3530.21o	22,781 KB
BRST3540.21o	22,509 KB
BRST3550.21o	22,279 KB
BRST3560.21o	22,567 KB
BRST3570.21o	22,982 KB
BRST3580.21o	22,783 KB
gbm21885.sp3	2,828 KB
gbm21886.sp3	2,828 KB
gbm21890.sp3	2,805 KB
gbm21891.sp3	2,805 KB
gbm21892.sp3	2,850 KB
gbm21893.sp3	2,828 KB
gbm21894.sp3	2,828 KB
gbm21895.sp3	2,805 KB
gbm21896.sp3	2,805 KB

Figure 4. Observation files (RINEX 3) and navigation files (precise ephemeris) of test data.

First step: Reflected Signals extraction

The first step is to run the '*crenameList.m*' script, which generates a series of file lists based on the existing observation and navigation files. For '*crenameList.m*', user needs to specify the path of the observation file, the path of the navigation file, and the output file path to storing the generated files (see Figure 5).

```
function creNameList(ofilepath,nfilepath,outputfilepath)
%% Create namelist files
% Inputs :
% ofilepath : the path of Obversation FILES(RINEX 3)
% nfilepath : the path of Navigation FILES(sp3)
% outputfilepath : the path of Output FILES(namelist files)
% the namelist files will be stored in 'outputfilepath'
% eg.
% ofilepath = '..\Data\o\';
% nfilepath = '..\Data\n\';
% outputfilepath = '..\Data\output\';
% creNameList(ofilepath,nfilepath,outputfilepath);
```

Figure 5. The '*crenameList.m*' function.

As shown in Figure 6, after executing the script, five name list files will be created in the output folder. The files '*anavFile.txt*', '*bnavFile.txt*', and '*nnavFile.txt*' contain the paths and filenames of the navigation files, while '*obsFile.txt*' corresponds to the observation files. Check the contents of these files to ensure there are consistent with the input files. '*outFile.txt*' contains the paths and filenames of the output files will be generated in first step.

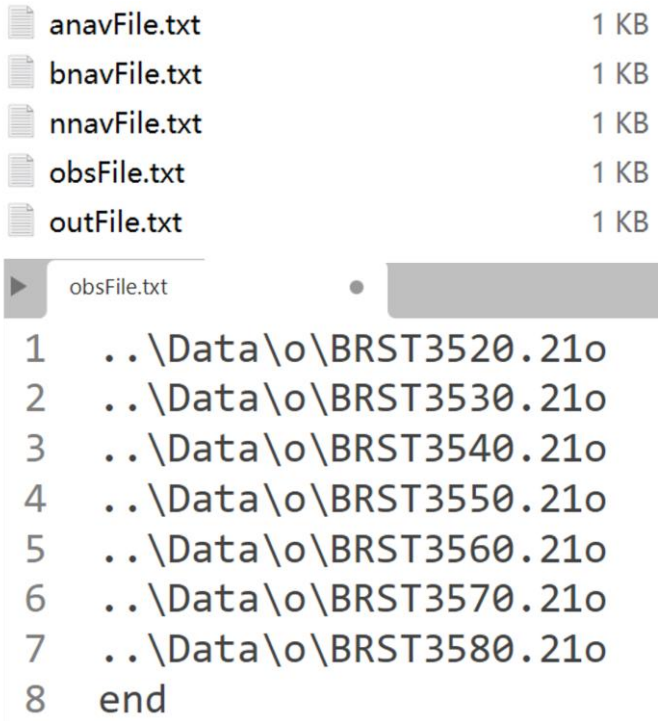

Figure 6. The name list files created by '*crenameList.m*' function, and the content of '*obsFile.txt*'.

After verifying these files, modify the content of configuration file based on the constellations and observation codes available in the RINEX file. As shown in Figure 7, the configuration file (in '*GNSS-R*' folder), '*GNSS.par*', contains 4 rows and 17 columns.

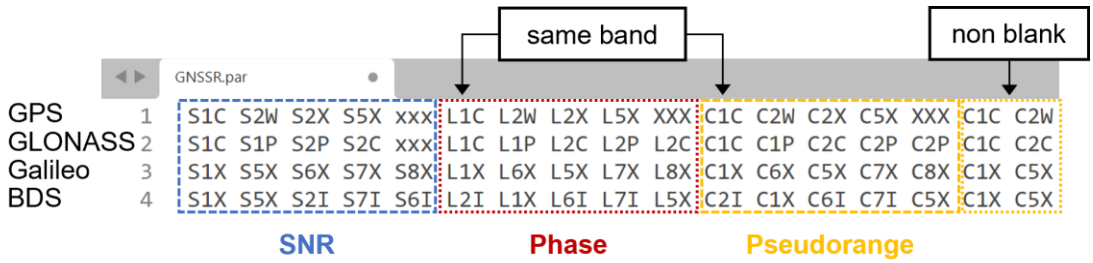

Figure 7. The configuration file, '*GNSS.par*', contains 4 rows and 17 columns, with each element corresponding to an observation code from the observation files or blank(xxx/XXX).

Rows 1 to 4 correspond to the four constellations: GPS, GLONASS, Galileo, and BDS. Columns 1 to 5 represent the SNR observation type; columns 6 to 10 correspond to phase observation type; and columns 11 to 15 correspond to pseudorange observation type. Each element is a 3-character observation code, which could be found in the observation files. Please ensure that the selected observation codes are available in each RINEX file. The empty fields can be replaced with 'XXX' or 'xxx'. For columns 16 and 17, users should select two pseudorange observation codes, which will be used to calculate the elevation angle. Both identical and different codes are acceptable, but these two columns should not be left blank.

It should be noted that when using the CMC method, the selected phase observations and pseudorange observations for reflector height inversion must correspond in order and belong to the same frequency band (the second character of observation code represents the frequency band). For example, in the first row of Figure 7, the first phase observation code (column 6) is 'L1C', then the corresponding first pseudorange observation code (column 11) should be 'C1C', not 'C2C'. In addition, for reflector height retrieval based on the CMC method, the values of the linear fitting parameters depend on the frequency band, and the specific values are given in Zhang et al. (2022).

After completing the configuration file according to your

requirements, copy it to the path of the file lists (for this example: the '*output*' folder). The software will then extract the selected signals based on the input files and produce '**.SlcData*' files (see Figure 8). There are two ways to implement this: the first method is to directly run the '*SNR.cpp*' script located in the '*GNSS-R*' folder, which requires the installation of a C++ compiler; while the second method (the preferred method) is to execute the MEX file '*mymex.mexw64*' located in the '*GNSS-R*' folder using MATLAB.

GNSSR.par	1 KB
BRST3520.21.SlcData	16,377 KB
BRST3530.21.SlcData	16,690 KB
BRST3540.21.SlcData	16,395 KB
BRST3550.21.SlcData	16,406 KB
BRST3560.21.SlcData	16,518 KB
BRST3570.21.SlcData	16,893 KB
BRST3580.21.SlcData	16,742 KB
anavFile.txt	1 KB
bnavFile.txt	1 KB
nnavFile.txt	1 KB
obsFile.txt	1 KB
outFile.txt	1 KB

Figure 8. '**.SlcData*' files generated based on the input files and the configuration file.

For the first method, the software provides C++ source files and header files in the '*signalExtraction*' folder, allowing users to execute the program in a Windows environment (see Figure 9). In this example, Microsoft Visual Studio 2022 is used: open the '*SNR.sln*' file, modify the input file path in '*SNR.cpp*', and execute the program. The software will

generates '*.*SlcData*' files, as shown in Figure 8.

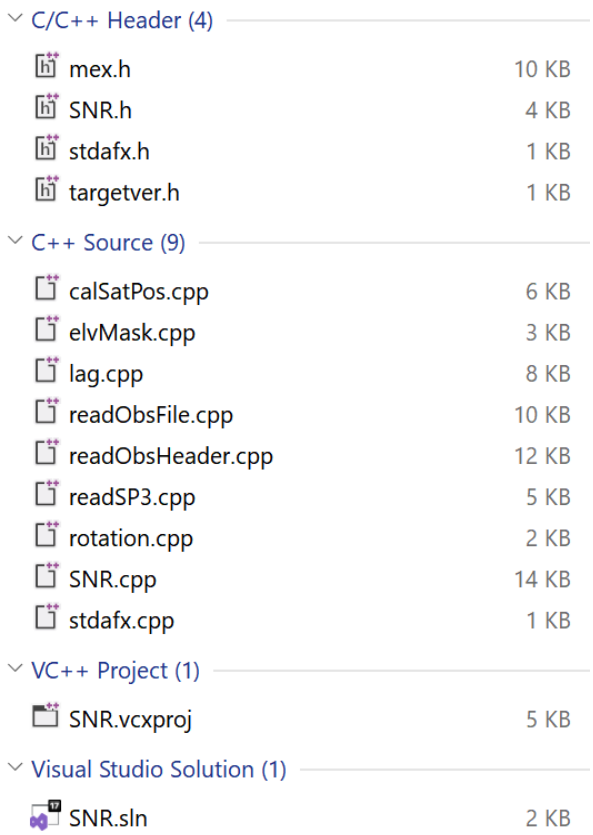

Figure 9. The software provided C++ source files and header files, which are used to generate '*.*SlcData*' files in a Windows environment

In addition, users can directly execute MEX files in MATLAB, which is the preferred method. A MEX file, created in MATLAB, behaves like a MATLAB script which could call a C/C++ program. The '*createMEXFile*' function in the '*MEX*' folder demonstrates how to create a MEX file based on the C++ source files provided, along with examples of using the MEX file (see Figure 10). Users could also edit the C++ source files and regenerate the MEX file. Please note that the '*SNR.cpp*' file in the '*MEX*' folder, which is used to generate the MEX file, contains

slight differences compared to version in '*SNR*' folder.

```
%% Create MEX file based on *.cpp files
%run the script in the path of *.cpp files
mex -output mymex *.cpp

%% Test MEX file
%%input the path of filelists
filepath = '..\Data\output\';
%%Ensure the configuration file is in the input path
mymex(filepath);
```

Figure 10. The '*createMEXFile*' function in the '*MEX*' folder.

In the first step, the software generates name list files based on the observation files and navigation files. Utilizing these files and configuration file, the software generate '**.SlcData*' files that can be applied in the subsequent step. Users could refer to the relevant content in the '*GNSSRtest.m*' script for further guidance, as illustrated in Figure 11.

```
%% First step
% Modify the input according to the path of test data
ofilepath = '..\Data\o\';
nfilepath = '..\Data\n\';
outputfilepath = '..\Data\output\';
creNameList(ofilepath,nfilepath,outputfilepath);

% Extract the selected signals : Produce '*.SlcData' files
% Run 'SNR.cpp'
% Run MEX file  (ensure the configuration file is in the input path)
namelistfilepath = '..\Data\output\';
mymex(namelistfilepath);
```

Figure 11. Relevant content in the '*GNSSRtest.m*' for the first step.

Second step: Reflector height calculation

The second step is to calculate the reflection height based on the files generated in the first step, using the main function '*rhCal.m*'.

‘*rhCal.m*’ requires the following inputs: constellation, observation type, azimuth angle range, elevation angle range, reflector height range, and reflection area identifier. Users can also refer to the ‘*rhCal.m*’ function for additional guidance. Figure 12 shows how to execute the ‘*rhCal.m*’. Below is the explanation of the input parameters:

(1) Constellation: the software supports four satellite constellations: GPS, GLONASS, Galileo and BDS. Ensure that the selected constellations are available in the RINEX files.

(2) Observation type: users could choose the SNR, CMC, or a combination of both (input = ’*SNRCMC*’). When applying the CMC method, the linear fitting parameter values can be adjusted in ‘*sigHeight*’ function according to the frequency band specified in the configuration file.

(3) Path of ‘**.SlcData*’ files: for this example, path could be set as ‘*..\Data\output*’. The software will create a new folder within the selected path to store the output files of the second step.

(4) ‘*areatag*’: the composition of reflection area. The software supports multi-part reflection regions.

(5) Range of elevation and azimuth angle: proper elevation and azimuth masks ensure the reflection zone is situated over water surface. Users can configure the elevation and azimuth masks according to the relative position of the GNSS antenna with respect to the surrounding

water body. Users could verify whether the selected reflection masks are appropriate by plotting the first Fresnel reflection zone of the GNSS receiver (Roesler and Larson 2018).

(6) Range of reflector height: the reflector height actually varies within a certain range, and values outside this range could be considered noise. For the BRST station, the reflection height can be set between 12 and 20 meters.

```
%% Second step
% Use the '*.SlcData' file generated in the first step
% Adjust the input according to your requirements
systype = 'GREC';
path = '..\Data\output\';
elemask = [5,20;12,25];
azimmask = [130,165;165,330];
areatag = 2;
rhmin = 12;
rhmax = 20;
obstype = 'SNR';
rhCal(systype,path,elemask,azimmask,areatag,rhmin,rhmax,obstype);
```

Figure 12. Relevant content in the '*GNSSRtest.m*' for the second step.

The software will generate a text file, with the filename composed of the site name, year, and day of year (DOY), such as '*BRST2021352.txt*'. As shown in Figure 13, the output file contains 13 columns. Users should pay attention to the third and ninth columns, which represent the reflector height and its corresponding time, respectively. Based on these two columns of data, a preliminary time series of sea surface height variations could be produced.

BRST2021352.txt

year	doy	maxRHL1	meanazm	sat	delT	minE	maxE	UTC	ObsType	p/k	DynCor	MaskPart
		m	deg		min	deg	deg	hr				
2021	352	16.40	137.64	G1	38	19.98	5.07	6.68	2	8.093	-0.532	1
2021	352	16.52	137.64	G1	38	19.98	5.07	6.68	3	11.235	-0.532	1
2021	352	16.50	137.64	G1	38	19.98	5.07	6.68	4	9.767	-0.532	1
2021	352	15.07	247.67	G1	33	24.81	12.13	0.89	3	10.649	0.831	2
2021	352	15.13	247.67	G1	33	24.81	12.13	0.89	4	11.122	0.831	2
2021	352	14.49	209.97	G3	29	24.87	12.21	2.27	3	10.746	0.732	2
2021	352	14.52	209.97	G3	29	24.87	12.21	2.27	4	10.308	0.732	2
2021	352	15.06	174.43	G4	29	24.85	12.19	4.18	2	9.337	0.729	2
2021	352	15.06	174.43	G4	29	24.85	12.19	4.18	3	9.986	0.729	2
2021	352	14.97	174.43	G4	29	24.85	12.19	4.18	4	10.397	0.729	2
2021	352	16.52	186.66	G5	30	24.89	12.13	13.52	2	7.999	-0.736	2
2021	352	16.52	186.66	G5	30	24.89	12.13	13.52	3	8.667	-0.736	2
2021	352	14.34	187.92	G6	28	24.82	12.15	9.74	2	9.656	-0.716	2

reflector height time

Figure 13. The output file of the second step.

The contents of each column are described below:

(1) '*year*' & '*doy*': year and DOY of the reflector height results. The software processes input files at daily time intervals, such as standard RINEX observation files, so avoid providing observation files that span multiple days.

(2) '*maxRHL1*'&' *UTC*': reflector height and its corresponding time.

(3) '*meanazm*', '*minE*' & '*maxE*': the average azimuth angle, minimum elevation angle, and maximum elevation angle of the satellite arc over the water surface.

(4) '*sat*': satellite PRN

(5) '*delT*': duration of the satellite arc over the water surface.

(6) '*ObsType*': observation type employed to calculate reflector height. 1-5 indicate the reflector height calculated using SNR, while 11-15 indicate reflection height calculated using CMC.

(7) '*p/k*': peak-to-noise ratio.

(8) '*DynCor*': dynamic correction parameter, which will be used in the third step.

(9) '*MaskPart*': corresponds to '*areatag*', referring to different reflection zones.

Several data processing functions ('*plomb.m*', '*fit.m*', and '*medfilt1.m*'.) are used during the computation process. Simplified versions of these functions are provided, allowing the software to operate properly without additional toolboxes. Users can also install the '*Curve Fitting Toolbox*' and '*Signal Processing Toolbox*' to enable the full functionality of these functions. If the above toolboxes are already installed, users could modify the corresponding 'flag' parameters in '*rhcal.m*', '*OutDect.m*', and '*dataAnalysis.m*'.

Third step: High-resolution sea level estimation

To obtain a high-resolution sea surface height sequence, the software performs outlier detection and applies the second-order dynamic model using a sliding window on the reflector height files.

The main function for the third step, '*dataAnalysis*', requires the following inputs:

(1) '*rhsystype*'&'*rhobstype*': satellite constellation and observation type. Ensure that all selected inputs are available in the reflector height files.

(2) '*startdoy*'&'*enddoy*': start DOY and end DOY of the processing

period.

(3) '*step*'&'*len*': step size (min) and window length (h) of the sliding window. For the BRST station, they can be set to 10 min and 4 h, respectively (Zhang et al. 2022). The specific values depend on the period of sea level variations, and the may be adjusted through multiple trials to determine the most appropriate values. The value of the step size determines the time resolution of the reflector height sequence.

(4) '*rhpath*': path of reflector height files. For this example, it could be '..*Data**output**DataAnalysis**BRST*'.

(5) '*order*': type of dynamic model. When the input is 3, the software considers the sea surface height, its vertical velocity, and vertical acceleration, using a second-order dynamic model. While the input is 2, it considers the sea surface height and its speed, using a first-order model.

The relevant content of the third step in the '*GNSSRtest.m*' function is illustrated as Figure 14:

```
%% Third step
% Use the reflector height file(txt) generated in the second step
% Adjust the input according to your requirements
rhsystyp = 'GREC';
rhobstype = 'SNR';
startdoy = 352;
enddoy = 358;
step = 10;
len = 4;
rhpath = '..\Data\output\DataAnalysis\BRST\';
order = 3;
dataAnalysis(rhsystyp, rhobstype, startdoy, enddoy, step, len, rhpath, order);
```

Figure 14. Relevant content in the '*GNSSRtest.m*' for the third step.

As shown in Figure 15, the software generates two text files. The file '*winRH.txt*' contains the final high-resolution sea surface height sequence and it includes two columns: time (day) and reflector height (m). Notably, the software emphasizes relative changes in SSH rather than absolute changes; therefore, all output results have been de-averaged. Additionally, the software also outputs another result file, '*outdectRH.txt*', which has only applied outlier detection and has not been corrected by the dynamic model.

Figure 15. The output file of the third step. '*winRH.tx*t' contains the final high-resolution sea surface height sequence.

After completing the data processing and generating results based on prompts of the '*GNSSRtest.m*' function, users can compare the output files with the reference results in '*output-reference*' folder. If the output matches the reference results, it indicates that the software is functioning properly.

GUI

The software also provides a graphical user interface (GUI), '*GNSSR.mlapp*', developed using MATLAB App Designer. Figure 16 shows the main interface of the GUI. The GUI includes three interfaces corresponding to the three functional modules of the software. The

normal operation of the GUI requires MATLAB App Designer. Users could refer to the previous sections for instructions on running the GUI.

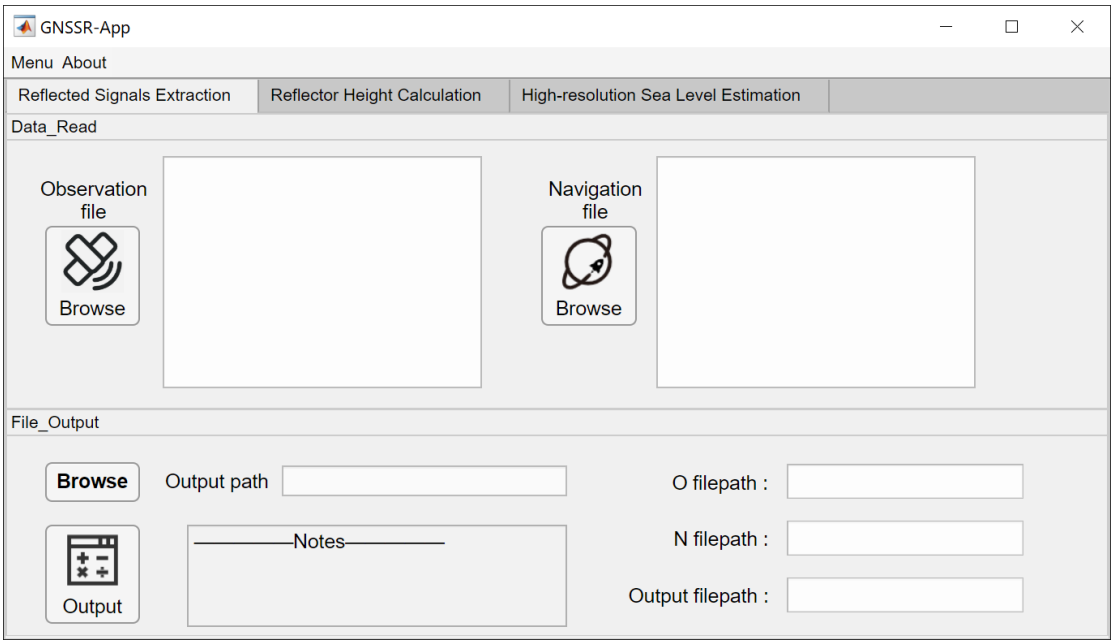

Figure 16. The MATLAB App '*GNSSR.mlapp*'.

In the 'Reflected Signals extraction' module, users begin by clicking the 'Browse' button to select the path of the observation files, the navigation files, and the output files. As shown in Figure 17, the information of selected paths and files will be displayed. After clicking the 'Output' button, the software generates output files, which are consistent with the results described in the first step.

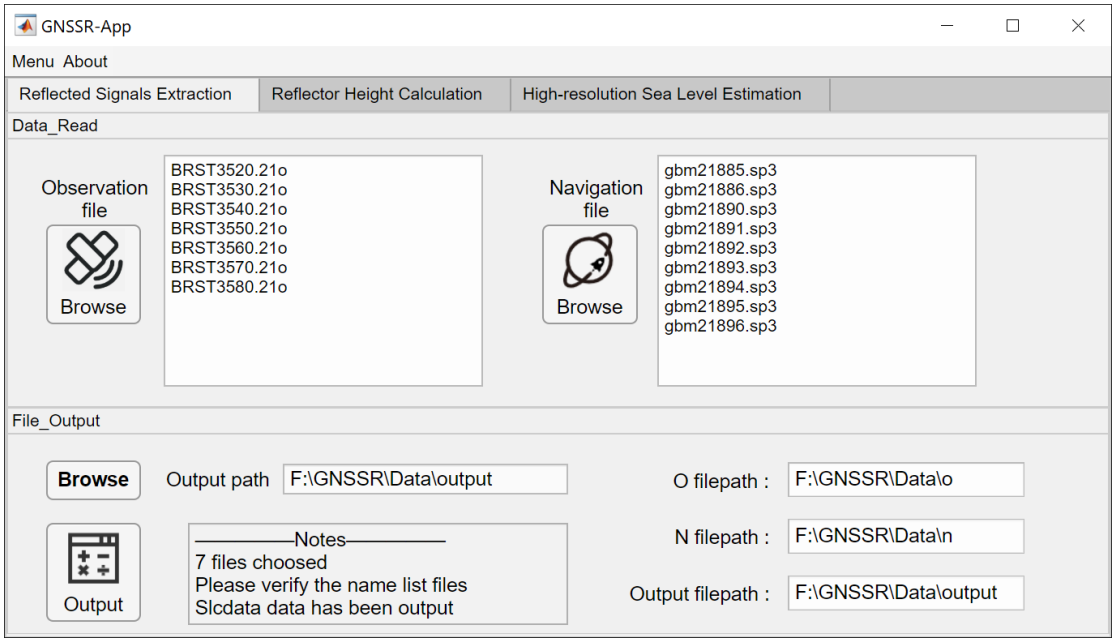

Figure 17. The 'Reflected Signals extraction' module in '*GNSSR.mlapp*'.

In the second interface, 'Reflector height calculation', users modify the displayed parameters displayed and select the output directory from the previous step. Ensure that the configuration file is located in the same directory. The GUI generates reflector height files after clicking the 'Calculate' button. Users could preview the output by clicking the 'Show' button and selecting the reflector height files (see Figure 18).

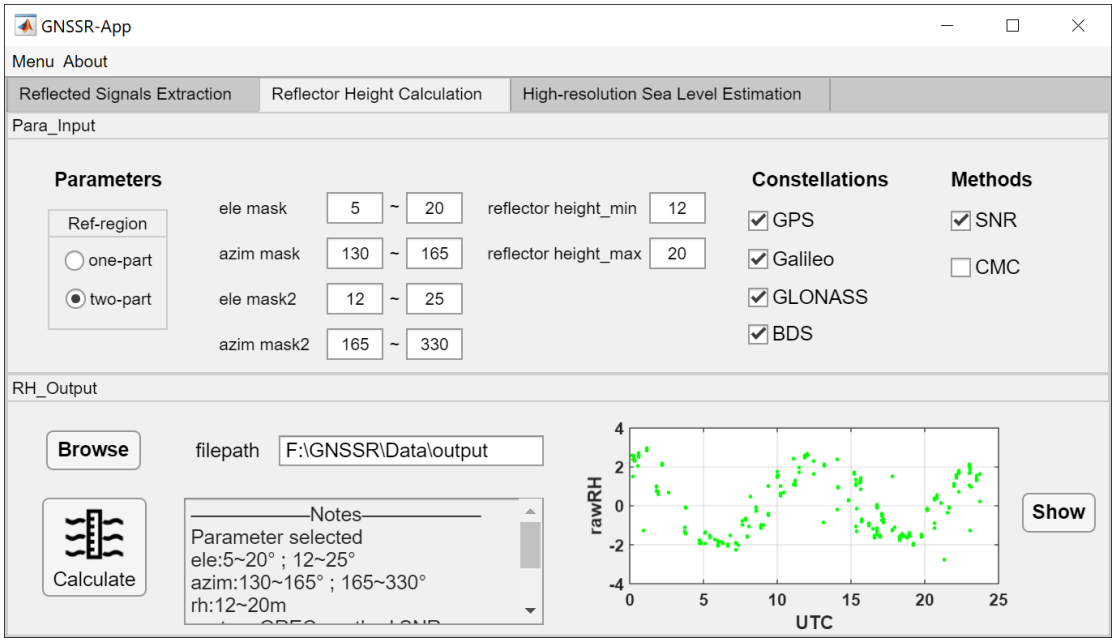

Figure 18. The 'Reflector Height calculation' module in '*GNSSR.mlapp*'.

In the 'High-resolution sea level estimation' module, users specify the path of reflector height files and relevant parameters such as window length and step size. After clicking the 'Output' button, the GUI generates two text files, as describes in section 'Third step: High-resolution sea level estimation'. These output files can be previewed by clicking the 'Show' button, and users could also save both figures by selecting the 'Save' button.

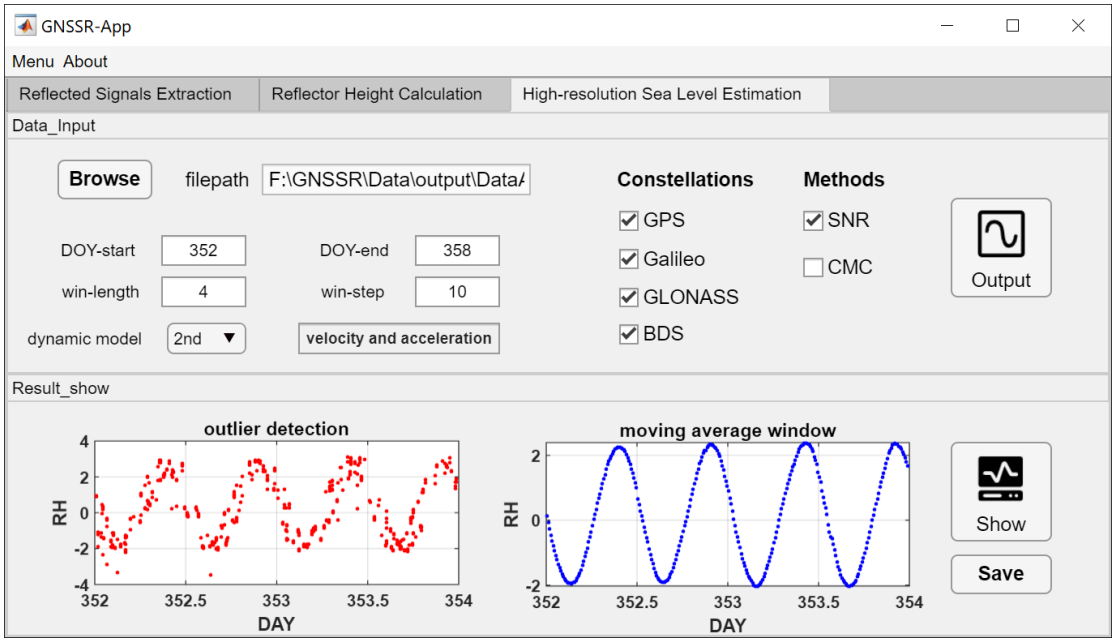

Figure 19. The 'High-resolution sea level estimation' module in '*GNSSR.mlapp*'.

References

Larson KM, Löfgren JS, Haas R (2013a) Coastal sea level measurements using a single geodetic GPS receiver. Adv Space Res 51(8):1301-1310. https://doi.org/10.1016/j.asr.2012.04.017

Larson KM, Ray RD, Nievinski, FG, Freymueller JT (2013b) The accidental tide gauge: A GPS reflection case study from Kachemak Bay, Alaska. IEEE Geosci Remote Sens Lett 10(5):1200-1204. https://doi.org/10.1109/LGRS.2012.2236075

Larson KM (2024) Gnssrefl: an open source software package in python for GNSS interferometric reflectometry applications. GPS Solut 28(4):165. https://doi.org/10.1007/s10291-024-01694-8

Martín A, Luján R, Anquela AB (2020). Python software tools for GNSS inter ferometric reflectometry (GNSS-IR). GPS Solut 24(4):94. https://doi.org/10. 1007/s10291-020-01010-0

Roesler C, Larson KM (2018) Software tools for GNSS interferometric reflectometry (GNSS-IR). GPS Solut 22(3):80. https://doi.org/10.1007/s10291-018-0744-8

Roussel N, Ramillien G, Frappart F, Darrozes J, Gay A, Biancale R, Striebig N, Hanquiez V, Bertin X, Allain D (2015) Sea level monitoring and sea state estimate using a single geodetic receiver. Remote Sens Environ 171:261–277. https://doi.org/10.1016/j.rse.2015.10.011

Wang X, He X, Zhang Q (2019) Evaluation and combination of quad-constellation multi-GNSS multipath reflectometry applied to sea level retrieval. Remote Sens Environ 231:111229. https://doi. org/ 10.1016/j.rse.2019.111229

Zhang Z, Guo F, Zhang X, Pan L (2022) First result of GNSS-R-based sea level retrieval with CMC and its combination with the SNR method. GPS Solut 26(1):20. https://doi.org/10.1007/s10291-021-01208-w